

Lecture 16:

C++ Multi-file Architecture

CEE 690

Gravity example revisited

```
#include <iostream>

int main() {
    // 1. Variable Declarations (The "Boxes" in memory)
    double y0 = 100.0;    // Initial height in meters
    double g = 9.81;      // Acceleration due to gravity
    double y;             // We will calculate this in the loop

    // 2. The Header Print
    std::cout << "Time(s) | Position(m)" << std::endl;
    std::cout << "-----" << std::endl;

    // 3. The Calculation Loop (0 to 5 seconds)
    for (int t = 0; t <= 2; t++) {

        // C++ Math: Note that we use 0.5 instead of 1/2
        // (In C++, 1/2 would be 0 because it's integer division!)
        y = y0 - (0.5 * g * t * t);

        // 4. Output the result
        std::cout << t << "      | " << y << std::endl;
    }

    return 0;
}
```

```
nc153 $ ./gravity.exe
Time(s) | Position(m)
-----
0        | 100
1        | 95.095
2        | 80.38
```

Split into two files (and more)

main.cpp

```
#include <iostream>

int main() {
    // Attempting to use the class defined in the other file
    GravityCalculator sim(100.0);

    std::cout << "Height at 2 seconds: " << sim.get_position(2.0) << "m" << std::endl;

    return 0;
}
```

gravity.cpp

```
#include <iostream>

class GravityCalculator {
private:
    double initial_height;
    double g = 9.81;

public:
    GravityCalculator(double h) {
        initial_height = h;
    }

    double get_position(double t) {
        return initial_height - (0.5 * g * t * t);
    }
};
```


It doesn't work...

```
[nc153 $ g++ main.cpp gravity.cpp -o gravity.exe
main.cpp: In function 'int main()':
main.cpp:5:5: error: 'GravityCalculator' was not declared in this scope
   5 |         GravityCalculator sim(100.0);
     |         ^~~~~~
main.cpp:7:45: error: 'sim' was not declared in this scope
   7 |         std::cout << "Height at 2 seconds: " << sim.get_position(2.0) << "m" << std::endl;
     |                                             ^~~
```

- The two files are compiled together but main.cpp has no idea what “GravityCalculator” is even if it was defined in gravity.cpp
- Basically, it is not “looking” in gravity.cpp for the definition of GravityCalculator
- We need to tell main.cpp what GravityCalculator looks like. But how?

Brute force fix

main.cpp

```
#include <iostream>

// --- THE BRUTE FORCE FIX ---
// We copy the "Blueprint" (the class structure) into this file
// so the compiler knows how much memory to allocate for 'sim'
class GravityCalculator {
private:
    double initial_height;
    double g = 9.81;

public:
    GravityCalculator(double h);           // We "promise" this exists
    double get_position(double t);        // We "promise" this exists
};
// -----

int main() {
    // Now the compiler says: "Aha! I know what a GravityCalculator is!"
    GravityCalculator sim(100.0);

    std::cout << "Height at 2 seconds: " << sim.get_position(2.0) << "m" << std::endl;

    return 0;
}
```

It works!

```
[nc153 $ g++ -c main.cpp -o main.o
[nc153 $ g++ -c gravity.cpp -o gravity.o
[nc153 $ g++ main.o gravity.o -o gravity.exe
[nc153 $ ./gravity.exe
Height at 2 seconds: 80.38m
```

Feature	<code>g++ -c main.cpp</code>	<code>g++ main.cpp</code>
Output	<code>main.o</code> (Object File)	<code>a.out</code> (Executable)
Runnable?	No (It's just a fragment)	Yes (It's a complete program)
Requires <code>main()</code> ?	No	Yes
The "Linker" Step	Skipped	Included
Analogy	Baking a single layer of a cake.	Baking all layers and frosting them together.

The `-c` allows us to work on file “by parts” and then we can pull everything together later on

Or you can just everything together from the get go

```
[nc153 $ g++ main.cpp gravity.cpp -o gravity.exe  
[nc153 $ ./gravity.exe  
Height at 2 seconds: 80.38m
```

With large pieces of software this becomes highly impractical for multiple reasons so we end up doing the “by parts” approach most frequently.

This is still not ideal though.. Why?

```
#include <iostream>
main.cpp
```

```
// --- THE BRUTE FORCE FIX ---
// We copy the "Blueprint" (the class structure) into this file
// so the compiler knows how much memory to allocate for 'sim'
class GravityCalculator {
private:
    double initial_height;
    double g = 9.81;

public:
    GravityCalculator(double h);
    double get_position(double t);
};
// -----

int main() {
    // Now the compiler says: "Aha! I know how much memory to allocate for 'sim'"
    GravityCalculator sim(100.0);

    std::cout << "Height at 2 seconds: ";

    return 0;
}
```

```
#include <iostream>
gravity.cpp
```

```
// We don't need the whole class block here if we use the preprocessor
// to define the specific functions
class GravityCalculator {
    double initial_height;
    double g = 9.81;
public:
    GravityCalculator(double h);
    double get_position(double t);
};

GravityCalculator::GravityCalculator(double h) {
    initial_height = h;
}

double GravityCalculator::get_position(double t) {
    return initial_height - (0.5 * g * t * t);
}
```


Header files (.hpp) are our friends

```
#ifndef GRAVITY_HPP
#define GRAVITY_HPP

class GravityCalculator {
private:
    double initial_height;
    double g = 9.81;

public:
    // We only put the signatures (the promises) here
    GravityCalculator(double h);
    double get_position(double t);
};

#endif
```

#ifndef - Has GRAVITY_HPP been defined already? If it has then don't add everything below again

#define - Define GRAVITY_HPP here

#endif - It closes the #ifndef scope/clause

Or just use #pragma once

```
#pragma once

class GravityCalculator {
private:
    double initial_height;
    double g = 9.81;

public:
    // We only put the signatures (the promises) here
    GravityCalculator(double h);
    double get_position(double t);
};
```

#pragma once - It is a single line substitute for #ifndef, #define, and #endif

Include .hpp file in .cpp files

main.cpp

```
#include <iostream>
#include "gravity.hpp" // Everything main needs to know is in here

int main() {
    GravityCalculator sim(100.0);
    std::cout << "Height at 2 seconds: " << sim.get_position(2.0) << "\n";
    return 0;
}
```

gravity.cpp

```
#include "gravity.hpp"

// Use the Scope Resolution Operator (::) to fulfill
GravityCalculator::GravityCalculator(double h) {
    initial_height = h;
}

double GravityCalculator::get_position(double t) {
    return initial_height - (0.5 * g * t * t);
}
```


It works!

```
[nc153 $ g++ -c main.cpp -o main.o  
[nc153 $ g++ -c gravity.cpp -o gravity.o  
[nc153 $ g++ main.o gravity.o -o gravity.exe  
[nc153 $ ./gravity.exe  
Height at 2 seconds: 80.38m
```

- All the header information is being “seen” by main.cpp and gravity.cpp.
- But happens if the .hpp file is not in the same place that you are compiling the code?

Let's put everything in a Makefile

```
# Step 1: Link the object files into the final program
gravity.exe: main.o gravity.o
    g++ main.o gravity.o -o gravity.exe

# Step 2: Compile the driver
main.o: main.cpp gravity.hpp
    g++ -c main.cpp

# Step 3: Compile the math
gravity.o: gravity.cpp gravity.hpp
    g++ -c gravity.cpp

# Step 4: Add a way to reset
clean:
    rm -f *.o gravity.exe
```

```
[nc153 $ make
g++ -c main.cpp
g++ -c gravity.cpp
g++ main.o gravity.o -o gravity.exe
[nc153 $ ./gravity.exe
Height at 2 seconds: 80.38m
```

```
[nc153 $ make clean
rm -f *.o gravity.exe
```

Note how the dependencies of each file are being defined

“Professional” Makefile

```
# 1. SETTINGS (The "Knobs" of your project)
CXX      = g++
TARGET   = gravity.exe
OBJS     = main.o gravity.o

# 2. THE BUILD RULES
# Build the final executable
$(TARGET): $(OBJS)
    $(CXX) $(OBJS) -o $(TARGET)

# Compile main.o
main.o: main.cpp gravity.hpp
    $(CXX) -c main.cpp

# Compile gravity.o
gravity.o: gravity.cpp gravity.hpp
    $(CXX) -c gravity.cpp

# 3. UTILITIES
clean:
    rm -f $(OBJS) $(TARGET)
```

- Makefile can have variables/settings
- This allows us to customize the compiler, the settings, etc...

```
[nc153 $ make
g++ -c main.cpp
g++ -c gravity.cpp
g++ main.o gravity.o -o gravity.exe
[nc153 $ ./gravity.exe
Height at 2 seconds: 80.38m
```


Detour: Compiler flags

- By default, the compiler (e.g., g++ will do the bare minimum to make a working program
- We can adjust the settings to demand higher quality, faster math, access to external tools, etc...

```
g++ [flags] source.cpp -o output
```

Safety flags

- **-Wall:** It tells the compiler to tell us about everything that looks suspicious (errors and warnings)
- **-Wextra:** Turns on even more niche warnings
- **-g:** Add debug symbols so that tools like GDB or Valgrind (we will talk about this in a couple of weeks) can tell us exactly which line of code caused a crash

Standard flag

- **-std=c++17:** Currently the sweet spot for compatibility and modern features. Other options include c++11, c++14, c++20...

Speed flags

- **-O0**: This is the default. There is no optimization; it compiles fast but runs “slow”.
- **-O2**: The standard for production. It rearranges your math to be more effective without changing the results.
- **-O3**: Aggressive optimization. It tries everything it can to speed up the code.
- **-march=native**: Tells the compiler to look at the exact CPU and use its specific hardware shortcuts

Finder flags

- **-I** (Include): Tells the compiler where the .hpp files are located.
 - Example: -I/usr/local/include
- **-L**: Tells the linker where the library files (.so and .a) are located.
 - Example: -L/usr/local/lib
- **-l**: Tells the linker which specific library to grab (e.g., we might have multiple netCDF)
 - Example: -lnetcdf (this looks for libnetcdf.so)

Makefile with flags

```
# 1. SETTINGS (The "Knobs" of your project)
```

```
CXX      = g++
CXXFLAGS = -Wall -O2 -std=c++17
TARGET   = gravity.exe
OBJS     = main.o gravity.o
```

```
# 2. THE BUILD RULES
```

```
# Build the final executable
```

```
$(TARGET): $(OBJS)
    $(CXX) $(OBJS) -o $(TARGET)
```

```
# Compile main.o
```

```
main.o: main.cpp gravity.hpp
    $(CXX) $(CXXFLAGS) -c main.cpp
```

```
# Compile gravity.o
```

```
gravity.o: gravity.cpp gravity.hpp
    $(CXX) $(CXXFLAGS) -c gravity.cpp
```

```
# 3. UTILITIES
```

```
clean:
    rm -f $(OBJS) $(TARGET)
```

```
[nc153 $ make
g++ -Wall -O2 -std=c++17 -c main.cpp
g++ main.o gravity.o -o gravity.exe
[nc153 $ make clean
rm -f main.o gravity.o gravity.exe
```

Static libraries

- **.a files:** You can store packages/libraries in .a files (compiled machine code) and then add them to your software on compilation.
- **Part of the software:** It is merged “permanently” into your application. You don’t have to look for it every time, it is part of the executable.
- **Zero dependencies:** Completely self-contained executable. The resulting .exe is entirely self-sufficient
- **Version locking:** The program is frozen so if someone changes the libraries your executable will still work (until you recompile it).

Static libraries

```
CXX      = g++
CXXFLAGS = -Wall -std=c++17
LIBNAME  = ./lib/libgravity.a

# The final program depends on the LIBRARY, not the .cpp
gravity_static.exe: main.o $(LIBNAME)
    $(CXX) main.o -o gravity.exe -L./lib -lgravity

# Rule to create the static library
$(LIBNAME): gravity.o
    ar rcs $(LIBNAME) gravity.o

# Individual object files
main.o: main.cpp gravity.hpp
    $(CXX) $(CXXFLAGS) -c main.cpp

gravity.o: gravity.cpp gravity.hpp
    $(CXX) $(CXXFLAGS) -c gravity.cpp

clean:
    rm -f *.o *.a gravity.exe
```

```
[nc153 $ make
g++ -Wall -std=c++17 -c main.cpp
g++ -Wall -std=c++17 -c gravity.cpp
ar rcs ./lib/libgravity.a gravity.o
g++ main.o -o gravity.exe -L./lib -lgravity
```


Include directory

- **.hpp files:** The .hpp files of libraries (e.g., netcdf) will be found in a “include” directory.

```
[(cee690) nc153 $ tree
.
├── gravity.cpp
├── include
│   └── gravity.hpp
├── lib
│   └── libgravity.a
├── main.cpp
└── Makefile
```

Include: Update makefile

```
CXX      = g++
CXXFLAGS = -Wall -std=c++17
LIBNAME  = libgravity.a
LIB_DIR  = ./lib
INC_DIR  = ./include

# The final program depends on the LIBRARY, not the .cpp
gravity_static.exe: main.o $(LIB_DIR)/$(LIBNAME)
    $(CXX) main.o -o gravity.exe -I$(INC_DIR) -L$(LIB_DIR) -lgravity

# Rule to create the static library
$(LIB_DIR)/$(LIBNAME): gravity.o
    ar rcs $(LIB_DIR)/$(LIBNAME) gravity.o

# Individual object files
main.o: main.cpp $(INC_DIR)/gravity.hpp
    $(CXX) $(CXXFLAGS) -c main.cpp -I$(INC_DIR)

gravity.o: gravity.cpp $(INC_DIR)/gravity.hpp
    $(CXX) $(CXXFLAGS) -c gravity.cpp -I$(INC_DIR)

clean:
    rm -f *.o *.a gravity.exe
```

```
(cee690) nc153 $ make
g++ -Wall -std=c++17 -c main.cpp -I./include
g++ -Wall -std=c++17 -c gravity.cpp -I./include
ar rcs ./lib/libgravity.a gravity.o
g++ main.o -o gravity.exe -I./include -L./lib -lgravity
```

Dynamic libraries

- **.so files:** They are the standard for most applications. The library isn't "baked" into your executable.
- **Runtime use:** Your executable calls on the .so file on runtime. It helps keep the .exe small.
- **Swap-in upgrade:** If you update the .so library then the program can automatically use it (this is good and bad)
- **Runtime search:** Since it is not part of the executable, every time you run the program it has to know where it is. This gets tricky as we will see in the next lecture.

Dynamic libraries

```
CXX      = g++
# 1. ADDED -fPIC to the flags
CXXFLAGS = -Wall -std=c++17 -fPIC
# 2. CHANGED extension to .so
LIBNAME  = libgravity.so
LIB_DIR  = ./lib
INC_DIR  = ./include

# The final program depends on the LIBRARY, not the .cpp
# 3. ADDED the -Wl,-rpath flag for runtime searching
gravity_dynamic.exe: main.o $(LIB_DIR)/$(LIBNAME)
    $(CXX) main.o -o gravity.exe -I$(INC_DIR) -L$(LIB_DIR) -lgravity -Wl,-rpath,$(LIB_DIR)

# Rule to create the dynamic library
# 4. REPLACED 'ar' with 'g++ -shared'
$(LIB_DIR)/$(LIBNAME): gravity.o
    $(CXX) -shared -o $(LIB_DIR)/$(LIBNAME) gravity.o

# Individual object files
main.o: main.cpp $(INC_DIR)/gravity.hpp
    $(CXX) $(CXXFLAGS) -c main.cpp -I$(INC_DIR)

gravity.o: gravity.cpp $(INC_DIR)/gravity.hpp
    $(CXX) $(CXXFLAGS) -c gravity.cpp -I$(INC_DIR)

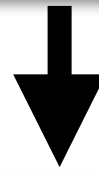
clean:
    # 5. UPDATED clean to target .so and the new exe name
    rm -f *.o $(LIB_DIR)/*.so gravity_dynamic.exe
```

```
[(cee690) nc153 $ make
g++ -Wall -std=c++17 -fPIC -c main.cpp -I./include
g++ -Wall -std=c++17 -fPIC -c gravity.cpp -I./include
g++ -shared -o ./lib/libgravity.so gravity.o
g++ main.o -o gravity.exe -I./include -L./lib -lgravity -Wl,-rpath,./lib
```

Rarely do we need to recompile everything when we change something

```
#include <iostream>
#include "gravity.hpp" // Everything main needs to know is in here

int main() {
    GravityCalculator sim(100.0);
    std::cout << "Height at 2 seconds: " << sim.get_position(2.0) << "m" << std::endl;
    return 0;
}
```



```
#include <iostream>
#include "gravity.hpp" // Everything main needs to know is in here

int main() {
    GravityCalculator sim(100.0);
    std::cout << "Height after 2 seconds: " << sim.get_position(2.0) << "m" << std::endl;
    return 0;
}
```

```
(cee690) nc153 $ make
g++ -Wall -std=c++17 -fPIC -c main.cpp -I./include
g++ main.o -o gravity.exe -I./include -L./lib -lgravity -Wl,-rpath,./lib
```